

开源软件基础

数据库 sqlite3

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

December 7, 2017



Outline

1 调研结果

2 Sqlite3

Outline

1 调研结果

2 Sqlite3

sqlite3 — DB-API 2.0 interface for SQLite databases

Source code `Lib/sqlite3/`

sqlite3 — DB-API 2.0 interface for SQLite databases

SQLite is a C library that provides a lightweight disk-based database that doesn't require a separate server process and allows accessing the database using a nonstandard variant of the SQL query language. Some applications can use SQLite for internal data storage. It's also possible to prototype an application using SQLite and then port the code to a larger database such as PostgreSQL or Oracle.

sqlite3 — DB-API 2.0 interface for SQLite databases

The sqlite3 module was written by Gerhard Häring. It provides a SQL interface compliant with the DB-API 2.0 specification described by PEP 249.

sqlite3 — DB-API 2.0 interface for SQLite databases

To use the module, you must first create a Connection object that represents the database. Here the data will be stored in the example.db file:

sqlite3 — DB-API 2.0 interface for SQLite databases

```
import sqlite3  
conn = sqlite3.connect('example.db')
```

sqlite3 — DB-API 2.0 interface for SQLite databases

You can also supply the special name `:memory:` to create a database in RAM.

sqlite3 — DB-API 2.0 interface for SQLite databases

Once you have a Connection, you can create a Cursor object and call its `Cursor.execute` method to perform SQL commands:

sqlite3 — DB-API 2.0 interface for SQLite databases

```
c = conn.cursor()

# Create table
c.execute('''CREATE TABLE stocks
            (date text, trans text, symbol text, qty
            # Insert a row of data
c.execute("INSERT INTO stocks VALUES ('2006-01-05','BU
            # Save (commit) the changes
conn.commit()

# We can also close the connection if we are done with
```

sqlite3 — DB-API 2.0 interface for SQLite databases

The data you've saved is persistent and is available in subsequent sessions:

sqlite3 — DB-API 2.0 interface for SQLite databases

```
import sqlite3
conn = sqlite3.connect('example.db')
c = conn.cursor()
```

sqlite3 — DB-API 2.0 interface for SQLite databases

Usually your SQL operations will need to use values from Python variables. You shouldn't assemble your query using Python's string operations because doing so is insecure; it makes your program vulnerable to an SQL injection attack (see <https://xkcd.com/327/> for humorous example of what can go wrong).

sqlite3 — DB-API 2.0 interface for SQLite databases

Instead, use the DB-API's parameter substitution. Put `?` as a placeholder wherever you want to use a value, and then provide a tuple of values as the second argument to the cursor's `Cursor.execute` method. (Other database modules may use a different placeholder, such as `%s` or `:1`.) For example:

sqlite3 — DB-API 2.0 interface for SQLite databases

```
# Never do this -- insecure!
symbol = 'RHAT'
c.execute("SELECT * FROM stocks WHERE symbol = '%s'" %

# Do this instead
t = ('RHAT',)
c.execute('SELECT * FROM stocks WHERE symbol=?', t)
print(c.fetchone())

# Larger example that inserts many records at a time
purchases = [('2006-03-28', 'BUY', 'IBM', 1000, 45.00)
              ('2006-04-05', 'BUY', 'MSFT', 1000, 72.00)
              ('2006-04-06', 'SELL', 'IBM', 500, 53.00)]
```

sqlite3 — DB-API 2.0 interface for SQLite databases

To retrieve data after executing a `SELECT` statement, you can either treat the cursor as an iterator, call the cursor's `Cursor.fetchone` method to retrieve a single matching row, or call `Cursor.fetchall` to get a list of the matching rows.

sqlite3 — DB-API 2.0 interface for SQLite databases

This example uses the iterator form:

sqlite3 — DB-API 2.0 interface for SQLite databases

```
>>> for row in c.execute('SELECT * FROM stocks ORDER BY  
    print(row)
```

```
('2006-01-05', 'BUY', 'RHAT', 100, 35.14)
```

```
('2006-03-28', 'BUY', 'IBM', 1000, 45.0)
```

```
('2006-04-06', 'SELL', 'IBM', 500, 53.0)
```

```
('2006-04-05', 'BUY', 'MSFT', 1000, 72.0)
```

类型对应

SQLite 默认支持如下类型: NULL, INTEGER, REAL, TEXT, BLOB.

Table 1: 与 Python 类型间对应关系

Python type	SQLite type
None	NULL
int	INTEGER
float	REAL
str	TEXT
bytes	BLOB